

Lecture 0: Functions & Variables

Comprehensive Syntax Reference, Execution Models, and Method Analysis

Course: CS50's Introduction to Programming with Python

Target Track: Core Syntax & Placement Preparation

Language: Python 3

1. Program Execution, Comments & Basic I/O

Python programs execute top-to-bottom sequentially. Python uses standard indentation blocks (4 spaces) rather than curly braces `{}` to demarcate blocks.

Comment Conventions

Unlike C, C++, or Java which use `//` and `/* */`, Python utilizes the hash symbol `#` for inline comments.

```
# Single line comment name =  
input("What's your name? ")
```

Input Type Semantics

The `input()` builtin always captures data from standard input strictly as a text sequence: `str`.

```
# Returns type <class 'str'>  
user_val = input("Value: ")
```

2. `print()` Keyword Arguments: `sep` vs. `end`

By default, Python's `print(*objects, sep=' ', end='\n')` adds a space between multiple items and appends a trailing newline character.

Separator:

sep

Defines what goes between comma-separated arguments.

```
print("hello", name, sep="???") #  
Output: hello???Alice
```

Terminator:

end

Defines what is printed at the end of the statement.

```
print("hello", end="") # Prevents  
newline; keeps cursor inline
```

Syntax Note (Comma vs. Concatenation): Passing multiple items separated by commas `print("hello", name)` handles any object type cleanly and injects spaces. In contrast, string concatenation `"hello " + name` requires explicit spacing and will crash with a `TypeError` if applied to non-string variables without casting.

3. String Transformations & Queries

Method	Behavior	Example Output
<code>.strip()</code>	Removes leading and trailing whitespace.	<code>" cs50 ".strip()</code> → <code>"cs50"</code>
<code>.capitalize()</code>	Capitalizes only the very first character; lowercases the rest.	<code>"alan turing".capitalize()</code> → <code>"Alan turing"</code>
<code>.title()</code>	Capitalizes the first character of each word.	<code>"alan turing".title()</code> → <code>"Alan Turing"</code>
<code>.upper()</code>	Converts every single character to full uppercase (Asked in lecture).	<code>"hello world".upper()</code> → <code>"HELLO WORLD"</code>

4. Method Chaining & Variable Unpacking

Methods can be evaluated sequentially in a single expression. Unpacking allows assigning sequence items to distinct variables simultaneously.

```
# Chaining strip and title in one step name = input("Enter name: ").strip().title() #
Unpacking via split() first, last = name.split(" ")
```

Runtime Pitfall: `first, last = name.split(" ")` expects exactly 2 items. If the user enters three names or multiple spaces, Python raises `ValueError: too many values to unpack`. Using bare `name.split()` handles variable contiguous whitespace.

5. Numeric Types & f-String Formatting Specifiers

Type Casting: `int()` vs `float()`

Must cast `input()` to perform arithmetic, otherwise `+` will perform string concatenation.

```
x = float(input("x: ")) y =
int(input("y: ")) z = x + y
```

Formatting: Commas & Decimals

Format specifiers inside f-strings format numeric presentation easily.

```
print(f"{z:,}") # 1,000,000 style
print(f"{r:.2f}") # Fixed 2 decimal points
```

Banker's Rounding Note: The builtin `round(number, ndigits)` implements round-half-to-even. For example, `round(14.5)` resolves to `14`, whereas `round(15.5)` resolves to `16` to eliminate statistical bias.

6. Functions, Parameters, Return & Scope Architecture

Custom procedures are declared using the `def` keyword. Structuring code inside a designated `main()` routine provides clean scoping and avoids hoisting errors.

Concept	Implementation Pattern	Core Purpose
Positional Argument	<code>def hello(to):</code>	Passes caller data into local variable <code>to</code> .
Default Argument	<code>def hello(to="world"):</code>	Allows optional invocation without arguments.
Return Value	<code>return num * num</code>	Sends evaluated result back to caller context.
Variable Scope	Variables inside functions	Variables exist only within their defining function body.

```
def main(): x = int(input("what is the value of x? ")) print("x squared is", square(x)) def
square(num=1): return num * num # Hands the computed integer back to main() main() # Invoked at the
bottom after all function signatures are registered
```

Lecture 0 Mastery Checklist:

- Cleaned user input with `.strip()`, `.title()`, and `.upper()`.
- Managed print output using `sep=""` and `end=""`.
- Cast string inputs to `int` and `float`; formatted output via `f"{val:,.2f}"`.
- Built reusable functions with default parameters and explicit `return` statements.